

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg.No: Hemalakshmi H
Name: 192571049

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

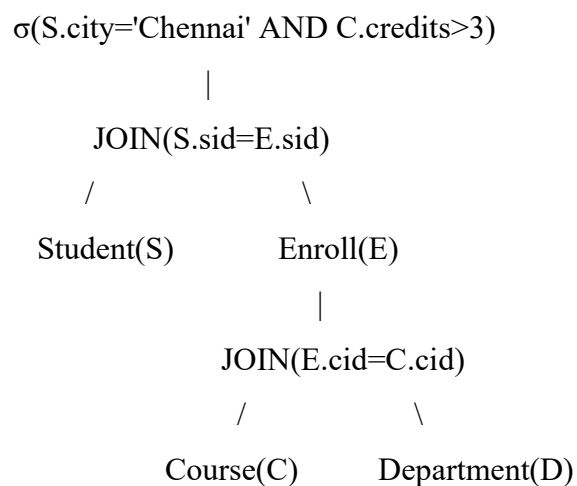
Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

1. Query Execution Plan + Heuristic Optimization

```
SELECT S.name, D.dept_name
FROM Student S
JOIN Enroll E ON S.sid = E.sid
JOIN Course C ON E.cid = C.cid
JOIN Department D ON C.dept_id = D.dept_id
WHERE S.city = 'Chennai' AND C.credits > 3;
```

A. Initial Query Tree



B. Apply Heuristic Optimization Rules

Rule 1: Push Selections Down

Code

$\sigma(S.city='Chennai') \rightarrow \text{Student}$

$\sigma(C.credits>3) \rightarrow \text{Course}$

Rule 2: Perform Projections Early

Keep only required attributes:

- Student: sid, name
- Course: cid, dept_id, credits

- Department: dept_id, dept_name
- Enroll: sid, cid

Rule 3: Reorder Joins to Reduce Intermediate Results

Join order based on smallest relations first:

1. Select Student (city='Chennai') → reduces Student drastically
2. Select Course (credits>3) → reduces Course
3. Join Enroll with filtered Student
4. Join result with filtered Course
5. Join with Department

Optimized Query Tree

```

      JOIN
    /   \
  JOIN   Department
 /   \
JOIN   Course(credits>3)
/   \
Student(city='Chennai') Enroll

```

2. Cost-Based Optimization

Given Sample Relation Sizes

Relation Tuples Pages

R 10,000 500

S 5,000 200

Cost Formulas

- **Nested Loop Join:** $\text{Cost} = \text{Pages}(R) + (\text{Tuples}(R) \times \text{Pages}(S)) = 500 + (10,000 \times 200) = \mathbf{2,000,500 \text{ I/Os}}$
- **Sort-Merge Join:** $\text{Cost} = \text{Sort}(R) + \text{Sort}(S) + \text{Merge Sort}(R) \approx 500 \log_2 500 \text{ Sort}(S) \approx 200 \log_2 200 \text{ Total} \approx \mathbf{\sim 5,000 \text{ I/Os}}$
- **Hash Join:** $\text{Cost} \approx \text{Pages}(R) + \text{Pages}(S) = 500 + 200 = \mathbf{700 \text{ I/Os}}$

Best Choice: Hash Join

Because it has the **lowest cost** and works well for large unsorted relations.

3. JDBC Connectivity + PreparedStatement

Java Code

```
java
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/school", "root", "password");

            String query = "SELECT * FROM student WHERE city = ?";
            PreparedStatement ps = con.prepareStatement(query);
            ps.setString(1, "Chennai");

            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                System.out.println(rs.getInt("sid") + " " + rs.getString("name"));
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

}
}
}

4. Conflict Serializability Using Precedence Graph

Given Schedule

Code

T1: R(X) W(Y)

T2: W(X) R(Y)

T3: R(Y)

Conflicts

- T1 R(X) vs T2 W(X) → edge T1 → T2
- T2 R(Y) vs T1 W(Y) → edge T2 → T1
- T2 R(Y) vs T3 R(Y) → no conflict
- T1 W(Y) vs T3 R(Y) → edge T1 → T3

Precedence Graph

Code

T1 → T2

T2 → T1 (cycle)

T1 → T3

Conclusion

Because **cycle exists**, schedule is **NOT conflict-serializable**.

5 . Basic 2PL + Deadlock Identification (5 Marks)

Given Transactions

Code

T1: Lock-X(A), Lock-X(B), Unlock(A), Unlock(B)

T2: Lock-X(B), Lock-X(A), Unlock(B), Unlock(A)

2PL Rules

- Growing phase → acquire locks
- Shrinking phase → release locks
- No new locks after unlock

Deadlock Scenario

- T1 locks A
- T2 locks B
- T1 requests B → waits
- T2 requests A → waits

Deadlock Resolution

Use **deadlock detection** → choose victim Kill **T2** (less work done) Release locks → allow T1 to continue.

6. Wait-Die & Wound-Wait

Assume Timestamps

- $TS(T1) = 10$
- $TS(T2) = 20$

Case: T2 requests lock held by T1

Wait-Die (older waits, younger dies)

- T2 (younger) requests lock held by T1 (older) → T2 **dies** and restarts.

Wound-Wait (older wounds younger)

- T1 (older) requests lock held by T2 (younger) → T2 **is rolled back** (wounded).

7. Immediate Update Recovery (UNDO/REDO) (5 Marks)

Sample Log

Code

<START T1>
<T1, X, 10, 20>
<START T2>
<T2, Y, 5, 15>
<COMMIT T1>
<CRASH>

Recovery Steps

- **UNDO uncommitted transactions:** T2 → undo Y = 5
- **REDO committed transactions:** T1 → redo X = 20

Final State

- X = 20
- Y = 5

8. Deferred Update (NO-UNDO / REDO) (5 Marks)

Sample Log

Code

<START T1>
<T1, A, 50>
<START T2>
<T2, B, 30>
<COMMIT T1>
<CRASH>

Rules

- No UNDO (updates not written before commit)
- Only REDO committed transactions

Recovery

- REDO T1 $\rightarrow A = 50$
- T2 not committed $\rightarrow$ ignore

9. Strict 2PL for Banking System (5 Marks)

Scenario

- T1: Transfer ₹500 from Account A to B
- T2: Read balance of A

Strict 2PL Rules

- All locks held until commit
- Prevents dirty reads

Locking Sequence

Code

T1: Lock-X(A), Lock-X(B)

T2: Lock-S(A) $\rightarrow$ must wait (because T1 holds X(A))

T1: $A = A - 500$; $B = B + 500$

T1: COMMIT $\rightarrow$ release locks

T2: Lock-S(A) $\rightarrow$ now allowed

T2: Read(A)

Result

No inconsistency, no dirty read.

10. Pseudocode for Heuristic Query Optimization (5 Marks)

Pseudocode

Code

function optimizeQuery(Q):

 parse Q into relational algebra tree

// Rule 1: Push Selections Down

for each selection σ in tree:

 move σ to lowest possible node

// Rule 2: Project Early

for each projection π :

 remove unnecessary attributes

 push π down near base tables

// Rule 3: Order Joins

joinList = extract all joins

sort joinList by estimated size of intermediate results

rebuild tree using sorted join order

return optimized tree